```
0000 WITH GRMR_TBL, GRMR_OPS;¶USE  GRMR_TBL, GRMR_OPS;¶SEPARATE(¬IDL )¶--|-----------------------------------------------------------------------------------------------------
     ----------------¶--|¬¬LOAD_GRMR¶--|-----------------------------------------------------------------------------------------------------¶PROCEDURE¬LOAD_
     GRMR¬( NOM_TEXTE :STRING¬) IS¶¶  PROCEDURE READ_PARSE_TABLES¬IS¶    PTFILE¬¬: FILE_TYPE;¶    PTCHAR¬¬: CHARACTER;¶    PTINDEX¬¬: INTEGER;¶    LAST¬¬¬:
      NATURAL;¶    NTER_TEXT¬¬: STRING(¬1 .. 256 );¶    NTER_TXTREP¬¬: TREE;¶    STRING_SEEN¬¬: BOOLEAN;¶    --|-----------------------------------------
     ----------------------------------------------------¶    --|¬PROCEDURE¬STORE_ACTION¶    PROCEDURE STORE_ACTION IS¶      ACTION¬¬: INTEGER;¶      ACTION_
     OP¬¬: GRMR_OP;¶      TXT¬¬¬: STRING(¬1 .. 10 );¶      LAST¬¬: INTEGER;¶      SYM¬¬¬: TREE;¶    BEGIN¶      STRING_SEEN := FALSE;¶      GRMR.AC_TBL_LAS
     T := PTINDEX;¶      INT_IO.GET ( PTFILE, ACTION );¶      IF ACTION < 0¬THEN¶        GRMR.AC_TBL( PTINDEX ) := AC_SHORT( ACTION );¶      ELSE¶        A
     CTION_OP := GRMR_OP'VAL( ACTION / 1000 );¶        IF ACTION_OP NOT IN GRMR_OP_QUOTE THEN¶¬GRMR.AC_TBL(PTINDEX) := AC_SHORT( ACTION );¶        ELSE¶¬GE
     T_LINE(¬PTFILE, TXT, LAST );¶¬SYM := STORE_SYM( TXT( 1 .. LAST ) );¶¬GRMR.AC_TBL( PTINDEX ) := AC_SHORT( ACTION + INTEGER( SYM.PG ) );¶¬GRMR.AC_TBL_LA
     ST :=¬GRMR.AC_TBL_LAST + 1;¶¬GRMR.AC_TBL( GRMR.AC_TBL_LAST¬) := AC_SHORT( SYM.LN );¶¬STRING_SEEN := TRUE;¶      END IF;¶      END¬IF;¶    END STORE_
0010 ACTION;¶¶  BEGIN¶    OPEN(¬PTFILE, IN_FILE, "parse.tbl" );¶    WHILE¬NOT END_OF_FILE( PTFILE ) LOOP¶      GET( PTFILE, PTCHAR );¶      INT_IO.GET( PTF
     ILE, PTINDEX );¶¶      IF PTCHAR = 'S' THEN¶        PUT( PTCHAR¬);¶        INT_IO.PUT(¬PTINDEX );¶        GRMR.ST_TBL_LAST := PTINDEX;¶        INT_IO.
     GET(¬PTFILE, GRMR.ST_TBL( PTINDEX ) );¶        INT_IO.PUT(¬GRMR.ST_TBL( PTINDEX ) );¶        NEW_LINE;¶¶      ELSIF PTCHAR = 'T' THEN¶        GRMR.AC_
     SYM_LAST := PTINDEX;¶        STORE_ACTION;¶        DECLARE¶¬I¬: INTEGER;¶        BEGIN¶¬INT_IO.GET ( PTFILE, I );¶¬GRMR.AC_SYM( PTINDEX ) := AC_BYTE(
     I );¶        END;¶        SKIP_LINE( PTFILE );¶      ELSIF PTCHAR = 'A' THEN¶        STORE_ACTION;¶        IF NOT STRING_SEEN THEN¶¬SKIP_LINE( PTFILE
     );¶        END IF;¶      ELSIF PTCHAR = 'N' THEN¶        PUT( PTCHAR¬);¶        INT_IO.PUT(¬PTINDEX );¶        GRMR.NTER_LAST := PTINDEX;¶        GET(
      PTFILE, PTCHAR );¬¬¬¬--| SAUTER L'ESPACE¶        GET_LINE( PTFILE, NTER_TEXT, LAST );¶        PUT_LINE( '¬' & NTER_TEXT( 1 ..¬LAST ) );¶        NTER_
     TXTREP¬:= STORE_TEXT ( NTER_TEXT( 1 .. LAST ) );¶        GRMR.NTER_PG( PTINDEX¬) := AC_BYTE( NTER_TXTREP.PG );¶        GRMR.NTER_LN( PTINDEX¬) := AC_B
     YTE( NTER_TXTREP.LN );¶      ELSE¶        PUT( PTCHAR¬);¶        INT_IO.PUT(¬PTINDEX );¶        PUT_LINE( "*****TABLE ERROR" );¶        RAISE PROGRAM_
     ERROR;¶      END¬IF;¶    END LOOP;¶    CLOSE( PTFILE );¶    PUT_LINE( "PARSE TABLES READ." );¶  END READ_PARSE_TABLES;¶¶  PROCEDURE WRITE_BINARY IS¶
0020   USE GRMR_TBL_IO;¶    BIN_FILE¬: GRMR_TBL_IO.FILE_TYPE;¶  BEGIN¶    CREATE( BIN_FILE, OUT_FILE, "parse.bin" );¶    WRITE( BIN_FILE, GRMR_TBL.GRMR );¶
       CLOSE( BIN_FILE¬);¶  END WRITE_BINARY;¶¶¶BEGIN¶  CREATE_IDL_TREE_FILE( NOM_TEXTE & ".lar" );¶  DECLARE¶    DUMMY¬¬: TREE¬:= STORE_SYM( "_ADDRESS")
     ;¶    USER_ROOT¬: TREE¬:= MAKE( DN_USER_ROOT );¶  BEGIN¶    D( XD_USER_ROOT, TREE_ROOT, USER_ROOT );¶    READ_PARSE_TABLES;¶    CLOSE_IDL_TREE_FILE;¶
       WRITE_BINARY;¶  END;¶--|-----------------------------------------------------------------------------------------------------¶END LOAD_GRMR;¶
```